MyCodeSchool

Exerciții — vectori în stil LeetCode

Recapitulare vectori · Erika · C++ · 10 probleme în stil LeetCode

Ce ai voie să folosești: vectori `int v[100]` + lungimea lor; funcții care primesc vectorul și lungimea; `int&` pentru o lungime care se schimbă sau pentru rezultate (ca la interclasare); un al doilea vector pentru rezultat; `for`/`while`, și imbricate; `if`/`else`; `%`; `swap`; `bool`; `return`.
Reține: lucrezi în `leetcode-vectori.h`, inclus în `Source.cpp` (întâi trebuie să compileze tot proiectul, vezi B1 din review). Problemele 1–10 se rezolvă în câte o funcție cu numele din enunț; pentru fiecare, o funcție `lc1()` … `lc10()` pune datele din *Input* în vector și afișează rezultatul exact ca în *Output*. Funcțiile problemelor întorc sau modifică vectorul, `cout` doar în `lcN()`. Rulezi **toate** exemplele, nu doar primul.
Încă NU folosim: `vector` din bibliotecă, `#include <algorithm>`, `sort`, `map`/`unordered_map`, `string`, `do...while`, vector de frecvență indexat după valoare (aici valorile pot fi negative). Nimic copiat din `algoritmi.h`.

Partea 1 — la fel ca la clasă

1. Running Sum of 1d Array

Easy

Funcția `runningSum` primește vectorul `nums` și îl transformă: pe fiecare poziție `i` ajunge suma elementelor de la poziția 0 până la `i`, inclusiv. Se modifică chiar vectorul primit.

```
Example 1
Input:  nums = [1,2,3,4]
Output: [1,3,6,10]
Explanation: [1, 1+2, 1+2+3, 1+2+3+4]

Example 2
Input:  nums = [1,1,1,1,1]
Output: [1,2,3,4,5]

Example 3
Input:  nums = [3,-1,2,-10,1]
Output: [3,2,4,-6,-5]
```

Constraints: `1 <= dim <= 100` · -10^6 `<= nums[i] <=` 10^6

2. Two Sum

Easy

Funcția `twoSum` primește vectorul `nums` și o valoare `target` și găsește pozițiile a două elemente diferite a căror sumă este `target`. Există exact o soluție; același element nu se folosește de două ori. Pozițiile se afișează în ordine crescătoare.

```
Example 1
Input:  nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: nums[0] + nums[1] == 9, deci [0,1].

Example 2
Input:  nums = [3,2,4], target = 6
Output: [1,2]

Example 3
Input:  nums = [-3,4,3,90], target = 0
Output: [0,2]
```

Constraints: `2 <= dim <= 100` · -10^6 `<= nums[i], target <=` 10^6 · există exact o soluție

3. Move Zeroes

Easy

Funcția `moveZeroes` mută toate zerourile din `nums` la final, păstrând ordinea celorlalte elemente. Lucrezi în același vector, fără un vector auxiliar.

```
Example 1
Input:  nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
Explanation: 1, 3, 12 rămân în ordinea lor; cele două zerouri
             ajung la final.

Example 2
Input:  nums = [0]
Output: [0]

Example 3
Input:  nums = [4,0,0,-2,7]
Output: [4,-2,7,0,0]
```

Constraints: `1 <= dim <= 100` · `-1000 <= nums[i] <= 1000`

Partea 2 — un pic mai grele

4. Find Pivot Index

Easy

Poziția pivot este poziția la care suma elementelor din stânga ei este egală cu suma elementelor din dreapta ei (elementul de pe poziție nu intră în nicio sumă). La marginea vectorului, suma din partea goală este 0.

Funcția `pivotIndex` întoarce cea mai din stânga poziție pivot, sau −1 dacă nu există.

```
Example 1
Input:  nums = [1,7,3,6,5,6]
Output: 3
Explanation: stânga: 1 + 7 + 3 = 11, dreapta: 5 + 6 = 11.

Example 2
Input:  nums = [1,2,3]
Output: -1

Example 3
Input:  nums = [2,1,-1]
Output: 0
```

Constraints: `1 <= dim <= 100` · `-1000 <= nums[i] <= 1000`

5. Best Time to Buy and Sell Stock

Easy

`prices[i]` este prețul unei acțiuni în ziua `i`. Cumperi o singură dată, într-o zi, și vinzi o singură dată, într-o zi de **după**. Funcția `maxProfit` întoarce cel mai mare profit posibil, sau `0` dacă nu se poate obține profit.

```
Example 1
Input:  prices = [7,1,5,3,6,4]
Output: 5
Explanation: cumperi în ziua 1 (preț 1), vinzi în ziua 4
             (preț 6): 6 - 1 = 5.

Example 2
Input:  prices = [7,6,4,3,1]
Output: 0

Example 3
Input:  prices = [2,4,1]
Output: 2
```

Constraints: `1 <= dim <= 100` · `0 <= prices[i] <= 10^4`

6. Squares of a Sorted Array

Easy

Vectorul `nums` este ordonat crescător și poate conține valori negative. Funcția `sortedSquares` construiește un al doilea vector cu pătratele elementelor, ordonat crescător, fără să sorteze.

```
Example 1
Input:  nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: pătratele sunt [16,1,0,9,100]; în ordine:
             [0,1,9,16,100].

Example 2
Input:  nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]

Example 3
Input:  nums = [-5,-5,-5]
Output: [25,25,25]
```

Constraints: `1 <= dim <= 100` · -10^4 `<= nums[i] <=` 10^4 · `nums` este ordonat crescător

7. Plus One

Easy

Vectorul `digits` conține cifrele unui număr mare, de la cea mai semnificativă la cea mai puțin semnificativă, fără zerouri la început. Funcția `plusOne` adună 1 la acest număr, în același vector; lungimea vectorului poate crește.

```
Example 1
Input:  digits = [1,2,3]
Output: [1,2,4]
Explanation: 123 + 1 = 124.

Example 2
Input:  digits = [9]
Output: [1,0]

Example 3
Input:  digits = [9,9,9]
Output: [1,0,0,0]
```

Constraints: `1 <= dim <= 99` · `0 <= digits[i] <= 9` · fără zerouri la început, în afară de numărul 0

Partea 3 — provocări

8. Product of Array Except Self

Medium

Funcția `productExceptSelf` construiește un al doilea vector în care, pe poziția `i`, se află produsul tuturor elementelor din `nums`, în afară de `nums[i]`. Nu ai voie să folosești împărțirea.

```
Example 1
Input:  nums = [1,2,3,4]
Output: [24,12,8,6]
Explanation: pe poziția 0: 2 * 3 * 4 = 24; pe poziția 1:
             1 * 3 * 4 = 12, și tot așa.

Example 2
Input:  nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]

Example 3
Input:  nums = [2,2,2]
Output: [4,4,4]
```

Constraints: `2 <= dim <= 100` · `-30 <= nums[i] <= 30` · orice produs încape într-un `int`

Follow-up: fără bucle imbricate.

9. Rotate Array

Medium

Funcția `rotate` rotește vectorul `nums` spre dreapta cu `k` pași: la fiecare pas, ultimul element trece pe prima poziție. Rezultatul rămâne în același vector.

```
Example 1
Input:  nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation: pasul 1: [7,1,2,3,4,5,6]
             pasul 2: [6,7,1,2,3,4,5]
             pasul 3: [5,6,7,1,2,3,4]

Example 2
Input:  nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]

Example 3
Input:  nums = [1,2], k = 5
Output: [2,1]
```

Constraints: `1 <= dim <= 100` · `-1000 <= nums[i] <= 1000` · `0 <= k <= 1000`

Follow-up: fără să muți vectorul pas cu pas de `k` ori.

10. Container With Most Water

Medium

`height[i]` este înălțimea unei linii verticale ridicate în punctul `i`. Alegi două linii; împreună cu axa orizontală formează un recipient, care ține *înălțimea liniei mai scurte × distanța dintre ele* unități de apă. Funcția `maxArea` întoarce cea mai mare cantitate de apă posibilă.

```
Example 1
Input:  height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: liniile de pe pozițiile 1 (8) și 8 (7):
             min(8, 7) * (8 - 1) = 49.

Example 2
Input:  height = [1,1]
Output: 1

Example 3
Input:  height = [4,3,2,1,4]
Output: 16
```

Constraints: `2 <= dim <= 100` · `0 <= height[i] <=` 10^4

Follow-up: o singură parcurgere, fără bucle imbricate.

Înainte să treci la problema următoare, rulezi funcția și pe (unde constrângerile le permit):

- un vector cu un singur element;
- un vector cu toate valorile egale;
- un vector cu valori negative.